

Secret

Secret

Secret 是一种包含少量敏感信息例如密码、令牌或密钥的对象。这样的信息可能会被放在 规约中或者镜像中。使用 Secret 意味着你不需要在应用程序代码中包含机密数据。

由于创建 Secret 可以独立于使用它们的 Pod，因此在创建、查看和编辑 Pod 的工作流程中暴露 Secret（及其数据）的风险较小。Kubernetes 和在集群中运行的应用程序也可以对 Secret 采取额外的预防措施，例如避免将敏感数据写入非易失性存储。

Secret 类似于 但专门用于保存机密数据。

默认情况下，Kubernetes Secret 未加密地存储在 API 服务器的底层数据存储（etcd）中。任何拥有 API 访问权限的人都可以检索或修改 Secret，任何有权访问 etcd 的人也可以。此外，任何有权限在命名空间中创建 Pod 的人都可以使用该访问权限读取该命名空间中的任何 Secret；这包括间接访问，例如创建 Deployment 的能力。

为了安全地使用 Secret，请至少执行以下步骤：

1. 为 Secret 启用静态加密。
2. 以最小特权访问 Secret 并启用或配置 RBAC 规则。
3. 限制 Secret 对特定容器的访问。
4. 考虑使用外部 Secret 存储驱动。

有关管理和提升 Secret 安全性的指南，请参阅 Kubernetes Secret 良好实践。

参见 Secret 的信息安全了解详情。

Secret 的使用 {#uses-for-secrets}

你可以将 Secret 用于以下场景：

- 设置容器的环境变量。
- 向 Pod 提供 SSH 密钥或密码等凭据。
- 允许 kubelet 从私有镜像仓库中拉取镜像。

Kubernetes 控制面也使用 Secret；例如，引导令牌 Secret 是一种帮助自动化节点注册的机制。

使用场景：在 Secret 卷中带句点的文件 {#use-case-dotfiles-in-a-secret-volume}

通过定义以句点 (.) 开头的主键，你可以“隐藏”你的数据。这些主键代表的是以句点开头的文件或“隐藏”文件。例如，当以下 Secret 被挂载到 secret-volume 卷上时，该卷中会包含一个名为 .secret-file 的文件，并且容器 dotfile-test-container 中此文件位于路径 /etc/secret-volume/.secret-file 处。

以句点开头的文件会在 ls -l 的输出中被隐藏起来；列举目录内容时必须使用 ls -la 才能看到它们。

使用场景：仅对 Pod 中一个容器可见的 Secret {#use-case-secret-visible-to-one-container-in-a-pod}

考虑一个需要处理 HTTP 请求，执行某些复杂的业务逻辑，之后使用 HMAC 来对某些消息进行签名的程序。因为这一程序的应用逻辑很复杂，其中可能包含未被注意到的远程服务器文件读取漏洞，这种漏洞可能会把私钥暴露给攻击者。

这一程序可以分隔成两个容器中的两个进程：前端容器要处理用户交互和业务逻辑，但无法看到私钥；签名容器可以看到私钥，并对来自前端的简单签名请求作出响应（例如，通过本地主机网络）。

采用这种划分的方法，攻击者现在必须欺骗应用服务器来做一些其他操作，而这些操作可能要比读取一个文件要复杂很多。

Secret 的替代方案 {#alternatives-to-secrets}

除了使用 Secret 来保护机密数据，你也可以选择一些替代方案。

下面是一些选项：

- 如果你的云原生组件需要执行身份认证来访问你所知道的、在同一 Kubernetes 集群中运行的另一个应用，你可以使用 ServiceAccount 及其令牌来标识你的客户端身份。
- 你可以运行的第三方工具也有很多，这些工具可以运行在集群内或集群外，提供机密数据管理。例如，这一工具可能是 Pod 通过 HTTPS 访问的一个服务，该服务在客户端能够正确地通过身份认证（例如，通过 ServiceAccount 令牌）时，提供机密数据内容。
- 就身份认证而言，你可以为 X.509 证书实现一个定制的签名者，并使用 CertificateSigningRequest 来让该签名者为需要证书的 Pod 发放证书。

- 你可以使用一个设备插件 来将节点本地的加密硬件暴露给特定的 Pod。例如，你可以将可信任的 Pod 调度到提供可信平台模块（Trusted Platform Module, TPM）的节点上。这类节点是另行配置的。

你还可以将如上选项的两种或多种进行组合，包括直接使用 Secret 对象本身也是一种选项。

例如：实现（或部署）一个，从外部服务取回生命期很短的会话令牌，之后基于这些生命期很短的会话令牌来创建 Secret。运行在集群中的 Pod 可以使用这些会话令牌，而 Operator 则确保这些令牌是合法的。这种责权分离意味着你可以运行那些不了解会话令牌如何发放与刷新的确切机制的 Pod。

Secret 的类型 {#secret-types}

创建 Secret 时，你可以使用 Secret 资源的 type 字段，或者与其等价的 kubectl 命令行参数（如果有的话）为其设置类型。Secret 类型有助于对 Secret 数据进行编程处理。

Kubernetes 提供若干种内置的类型，用于一些常见的使用场景。针对这些类型，Kubernetes 所执行的合法性检查操作以及对其所实施的限制各不相同。

- 内置类型：Opaque；用法：用户定义的任意数据
- 内置类型：kubernetes.io/service-account-token；用法：服务账号令牌
- 内置类型：kubernetes.io/dockercfg；用法：~/.dockercfg 文件的序列化形式
- 内置类型：kubernetes.io/dockerconfigjson；用法：~/.docker/config.json 文件的序列化形式
- 内置类型：kubernetes.io/basic-auth；用法：用于基本身份认证的凭据
- 内置类型：kubernetes.io/ssh-auth；用法：用于 SSH 身份认证的凭据
- 内置类型：kubernetes.io/tls；用法：用于 TLS 客户端或者服务器端的数据
- 内置类型：bootstrap.kubernetes.io/token；用法：启动引导令牌数据

通过为 Secret 对象的 type 字段设置一个非空的字符串值，你也可以定义并使用自己 Secret 类型（如果 type 值为空字符串，则被视为 Opaque 类型）。

Kubernetes 并不对类型的名称作任何限制。不过，如果你要使用内置类型之一，则你必须满足为该类型所定义的所有要求。

如果你要定义一种公开使用的 Secret 类型，请遵守 Secret 类型的约定和结构，在类型名前面添加域名，并用 / 隔开。例如：cloud-hosting.example.net/cloud-api-credentials。

Opaque Secret

当你未在 Secret 清单中显式指定类型时，默认的 Secret 类型是 Opaque。当你使用 `kubectl` 来创建一个 Secret 时，你必须使用 `generic` 子命令来标明要创建的是一个 Opaque 类型的 Secret。例如，下面的命令会创建一个空的 Opaque 类型的 Secret：

```
kubectl create secret generic empty-secret  
kubectl get secret empty-secret
```

输出类似于：

NAME	TYPE	DATA	AGE
empty-secret	Opaque	0	2m6s

DATA 列显示 Secret 中保存的数据条目个数。在这个例子中，0 意味着你刚刚创建了一个空的 Secret。

ServiceAccount 令牌 Secret {#service-account-token-secrets}

类型为 `kubernetes.io/service-account-token` 的 Secret 用来存放标识某的令牌凭据。这是为 Pod 提供长期有效 ServiceAccount 凭据的传统机制。

在 Kubernetes v1.22 及更高版本中，推荐的方法是通过使用 `TokenRequest` API 来获取短期自动轮换的 ServiceAccount 令牌。你可以使用以下方法获取这些短期令牌：

- 直接调用 `TokenRequest` API，或者使用像 `kubectl` 这样的 API 客户端。例如，你可以使用 `kubectl create token` 命令。
- 在 Pod 清单中请求使用投射卷挂载的令牌。Kubernetes 会创建令牌并将其挂载到 Pod 中。当挂载令牌的 Pod 被删除时，此令牌会自动失效。更多细节参阅启动使用服务账号令牌投射的 Pod。

只有在你无法使用 `TokenRequest` API 来获取令牌，并且你能够接受因为将永不过期的令牌凭据写入到可读的 API 对象而带来的安全风险时，才应该创建 ServiceAccount 令牌 Secret。更多细节参阅为 ServiceAccount 手动创建长期有效的 API 令牌。

使用这种 Secret 类型时，你需要确保对象的注解 `kubernetes.io/service-account-name` 被设置为某个已有的 ServiceAccount 名称。如果你同时创建 ServiceAccount 和 Secret 对象，应该先创建 ServiceAccount 对象。

当 Secret 对象被创建之后，某个 Kubernetes 会填写 Secret 的其它字段，例如 `kubernetes.io/service-account.uid` 注解和 `data` 字段中的 `token` 键值（该键包含一个身份认证令牌）。

下面的配置实例声明了一个 ServiceAccount 令牌 Secret：

创建了 Secret 之后，等待 Kubernetes 在 data 字段中填充 token 主键。

参考 ServiceAccount 文档了解 ServiceAccount 的工作原理。你也可以查看 Pod 资源中的 automountServiceAccountToken 和 serviceAccountName 字段文档，进一步了解从 Pod 中引用 ServiceAccount 凭据。

Docker 配置 Secret {#docker-config-secrets}

如果你要创建 Secret 用来存放用于访问容器镜像仓库的凭据，则必须选用以下 type 值之一来创建 Secret：

- `kubernetes.io/dockercfg`：存放 `~/.dockercfg` 文件的序列化形式，它是配置 Docker 命令行的一种老旧形式。Secret 的 data 字段包含名为 `.dockercfg` 的主键，其值是用 base64 编码的某 `~/.dockercfg` 文件的内容。
- `kubernetes.io/dockerconfigjson`：存放 JSON 数据的序列化形式，该 JSON 也遵从 `~/.docker/config.json` 文件的格式规则，而后者是 `~/.dockercfg` 的新版本格式。使用此 Secret 类型时，Secret 对象的 data 字段必须包含 `.dockerconfigjson` 键，其键值为 base64 编码的字符串包含 `~/.docker/config.json` 文件的内容。

下面是一个 `kubernetes.io/dockercfg` 类型 Secret 的示例：

如果你不希望执行 base64 编码转换，可以使用 `stringData` 字段代替。

当你使用清单文件通过 Docker 配置来创建 Secret 时，API 服务器会检查 data 字段中是否存在所期望的主键，并且验证其中所提供的键值是否是合法的 JSON 数据。不过，API 服务器不会检查 JSON 数据本身是否是一个合法的 Docker 配置文件内容。

你还可以使用 `kubectl` 创建一个 Secret 来访问容器仓库时，当你没有 Docker 配置文件时你可以这样做：

```
kubectl create secret docker-registry secret-tiger-docker \
  --docker-email=tiger@acme.example \
  --docker-username=tiger \
  --docker-password=pass1234 \
  --docker-server=my-registry.example:5000
```

此命令创建一个类型为 `kubernetes.io/dockerconfigjson` 的 Secret。

从这个新的 Secret 中获取 `.data.dockerconfigjson` 字段并执行数据解码：

```
kubectl get secret secret-tiger-docker -o jsonpath='{.data.*}' | base64 -d
```

输出等价于以下 JSON 文档（这也是一个有效的 Docker 配置文件）：

```
{
  "auths": {
    "my-registry.example:5000": {
      "username": "tiger",
      "password": "pass1234",
      "email": "tiger@acme.example",
      "auth": "dGlnZXI6cGFzc2EyMzQ="
    }
  }
}
```

auths 值是 base64 编码的，其内容被屏蔽但未被加密。任何能够读取该 Secret 的人都可以了解镜像库的访问令牌。

建议使用凭据提供程序来动态、安全地按需提供拉取 Secret。

基本身份认证 Secret {#basic-authentication-secret}

kubernetes.io/basic-auth 类型用来存放用于基本身份认证所需的凭据信息。使用这种 Secret 类型时，Secret 的 data 字段必须包含以下两个键之一：

- username：用于身份认证的用户名；
- password：用于身份认证的密码或令牌。

以上两个键的键值都是 base64 编码的字符串。当然你也可以在 Secret 清单中的使用 stringData 字段来提供明文形式的内容。

以下清单是基本身份验证 Secret 的示例：

Secret 的 stringData 字段不能很好地与服务器端应用配合使用。

提供基本身份认证类型的 Secret 仅仅是出于方便性考虑。你也可以使用 Opaque 类型来保存用于基本身份认证的凭据。不过，使用预定义的、公开的 Secret 类型

（kubernetes.io/basic-auth）有助于帮助其他用户理解 Secret 的目的，并且对其中存在的主键形成一种约定。

SSH 身份认证 Secret {#ssh-authentication-secrets}

Kubernetes 所提供的内置类型 kubernetes.io/ssh-auth 用来存放 SSH 身份认证中所需要的凭据。使用这种 Secret 类型时，你就必须在其 data（或 stringData）字段中提供一个 ssh-privatekey 键值对，作为要使用的 SSH 凭据。

下面的清单是一个 SSH 公钥/私钥身份认证的 Secret 示例：

提供 SSH 身份认证类型的 Secret 仅仅是出于方便性考虑。你可以使用 Opaque 类型来保存用于 SSH 身份认证的凭据。不过，使用预定义的、公开的 Secret 类型

（`kubernetes.io/tls`）有助于其他人理解你的 Secret 的用途，也可以就其中包含的主键名形成约定。Kubernetes API 会验证这种类型的 Secret 中是否设定了所需的主键。

SSH 私钥自身无法建立 SSH 客户端与服务器端之间的可信连接。需要其它方式来建立这种信任关系，以缓解“中间人（Man In The Middle）”攻击，例如向 ConfigMap 中添加一个 `known_hosts` 文件。

TLS Secret

`kubernetes.io/tls` Secret 类型用来存放 TLS 场合通常要使用的证书及其相关密钥。

TLS Secret 的一种典型用法是为 Ingress 资源配置传输过程中的数据加密，不过也可以用于其他资源或者直接在负载中使用。当使用此类型的 Secret 时，Secret 配置中的 `data`（或 `stringData`）字段必须包含 `tls.key` 和 `tls.crt` 主键，尽管 API 服务器实际上并不会对每个键的取值作进一步的合法性检查。

作为使用 `stringData` 的替代方法，你可以使用 `data` 字段来指定 base64 编码的证书和私钥。有关详细信息，请参阅 Secret 名称和数据的限制。

下面的 YAML 包含一个 TLS Secret 的配置示例：

提供 TLS 类型的 Secret 仅仅是出于方便性考虑。你可以创建 Opaque 类型的 Secret 来保存用于 TLS 身份认证的凭据。不过，使用已定义和公开的 Secret 类型

（`kubernetes.io/tls`）有助于确保你自己项目中的 Secret 格式的一致性。API 服务器会验证这种类型的 Secret 是否设定了所需的主键。

要使用 `kubect`l 创建 TLS Secret，你可以使用 `tls` 子命令：

```
kubect
```

l create secret tls my-tls-secret \
 --cert=path/to/cert/file \
 --key=path/to/key/file

公钥/私钥对必须事先存在，`--cert` 的公钥证书必须采用 .PEM 编码，并且必须与 `--key` 的给定私钥匹配。

启动引导令牌 Secret {#bootstrap-token-secrets}

`bootstrap.kubernetes.io/token` Secret 类型针对的是节点启动引导过程所用的令牌。其中包含用来为周知的 ConfigMap 签名的令牌。

启动引导令牌 Secret 通常创建于 kube-system 名字空间内，并以 bootstrap-token- 的形式命名；其中 `` 是一个由 6 个字符组成的字符串，用作令牌的标识。

以 Kubernetes 清单文件的形式，某启动引导令牌 Secret 可能看起来像下面这样：

启动引导令牌类型的 Secret 会在 data 字段中包含如下主键：

- token-id：由 6 个随机字符组成的字符串，作为令牌的标识符。必需。
- token-secret：由 16 个随机字符组成的字符串，包含实际的令牌机密。必需。
- description：供用户阅读的字符串，描述令牌的用途。可选。
- expiration：一个使用 RFC3339 来编码的 UTC 绝对时间，给出令牌要过期的时间。可选。
- usage-bootstrap-：布尔类型的标志，用来标明启动引导令牌的其他用途。
- auth-extra-groups：用逗号分隔的组名列表，身份认证时除被认证为 system:bootstrappers 组之外，还会被添加到所列的用户组中。

你也可以在 Secret 的 stringData 字段中提供值，而无需对其进行 base64 编码：

Secret 的 stringData 字段不能很好地与服务器端应用配合使用。

使用 Secret {#working-with-secrets}

创建 Secret {#creating-a-secret}

创建 Secret 有以下几种可选方式：

- 使用 kubectl
- 使用配置文件
- 使用 Kustomize 工具

对 Secret 名称与数据的约束 {#restriction-names-data}

Secret 对象的名称必须是合法的 DNS 子域名。

在为创建 Secret 编写配置文件时，你可以设置 data 与/或 stringData 字段。data 和 stringData 字段都是可选的。data 字段中所有键值都必须是 base64 编码的字符串。如果不希望执行这种 base64 字符串的转换操作，你可以选择设置 stringData 字段，其中可以使用任何字符串作为其取值。

data 和 stringData 中的键名只能包含字母、数字、-、_ 或 . 字符。stringData 字段中的所有键值对都会在内部被合并到 data 字段中。如果某个主键同时出现在 data 和 stringData 字段中，stringData 所指定的键值具有高优先级。

尺寸限制 {#restriction-data-size}

每个 Secret 的尺寸最多为 1MiB。施加这一限制是为了避免用户创建非常大的 Secret，进而导致 API 服务器和 kubelet 内存耗尽。不过创建很多小的 Secret 也可能耗尽内存。你可以使用资源配额来约束每个名字空间中 Secret（或其他资源）的个数。

编辑 Secret {#editing-a-secret}

你可以编辑一个已有的 Secret，除非它是不可变更的。要编辑一个 Secret，可使用以下方法之一：

- 使用 kubectl
- 使用配置文件

你也可以使用 Kustomize 工具编辑数据。然而这种方法会用编辑过的数据创建新的 Secret 对象。

根据你创建 Secret 的方式以及该 Secret 在 Pod 中被使用的方式，对已有 Secret 对象的更新将自动扩散到使用此数据的 Pod。有关更多信息，请参阅在 Pod 以文件形式使用 Secret。

使用 Secret {#using-a-secret}

Secret 可以以数据卷的形式挂载，也可以作为 暴露给 Pod 中的容器使用。Secret 也可用于系统中的其他部分，而不是一定要直接暴露给 Pod。例如，Secret 也可以包含系统中其他部分在替你与外部系统交互时要使用的凭证数据。

Kubernetes 会检查 Secret 的卷数据源，确保所指定的对象引用确实指向类型为 Secret 的对象。因此，如果 Pod 依赖于某 Secret，该 Secret 必须先于 Pod 被创建。

如果 Secret 内容无法取回（可能因为 Secret 尚不存在或者临时性地出现 API 服务器网络连接问题），kubelet 会周期性地重试 Pod 运行操作。kubelet 也会为该 Pod 报告 Event 事件，给出读取 Secret 时遇到的问题细节。

可选的 Secret {#restriction-secret-must-exist}

当你在 Pod 中引用 Secret 时，你可以将该 Secret 标记为**可选**，就像下面例子中所展示的那样。如果可选的 Secret 不存在，Kubernetes 将忽略它。

默认情况下，Secret 是必需的。在所有非可选的 Secret 都可用之前，Pod 的所有容器都不会启动。

如果 Pod 引用了非可选 Secret 中的特定键，并且该 Secret 确实存在，但缺少所指定的键，则 Pod 在启动期间会失败。

在 Pod 以文件形式使用 Secret {#using-secrets-as-files-from-a-pod}

如果你要在 Pod 中访问来自 Secret 的数据，一种方式是让 Kubernetes 将该 Secret 的值以文件的形式呈现，该文件存在于 Pod 中一个或多个容器内的文件系统内。

相关的指示说明，可以参阅创建一个可以通过卷访问 Secret 数据的 Pod。

当卷中包含来自 Secret 的数据，而对应的 Secret 被更新，Kubernetes 会跟踪到这一操作并更新卷中的数据。更新的方式是保证最终一致性。

对于以 subPath 形式挂载 Secret 卷的容器而言，它们无法收到自动的 Secret 更新。

Kubelet 组件会维护一个缓存，在其中保存节点上 Pod 卷中使用的 Secret 的当前主键和取值。你可以配置 kubelet 如何检测所缓存数值的变化。kubelet 配置中的 configMapAndSecretChangeDetectionStrategy 字段控制 kubelet 所采用的策略。默认的策略是 Watch。

对 Secret 的更新操作既可以通过 API 的 watch 机制（默认）来传播，基于设置了生命期的缓存获取，也可以通过 kubelet 的同步回路来从集群的 API 服务器上轮询获取。

因此，从 Secret 被更新到新的主键被投射到 Pod 中，中间存在一个延迟。这一延迟的上限是 kubelet 的同步周期加上缓存的传播延迟，其中缓存的传播延迟取决于所选择的缓存类型。对应上一段中提到的几种传播机制，延迟时长为 watch 的传播延迟、所配置的缓存 TTL 或者对于直接轮询而言是零。

以环境变量的方式使用 Secret {#using-secrets-as-environment-variables}

如果需要在 Pod 中以的形式使用 Secret：

1. 对于 Pod 规约中的每个容器，针对你要使用的每个 Secret 键，将对应环境变量添加到 env[].valueFrom.secretKeyRef 中。
2. 更改你的镜像或命令行，以便程序能够从指定的环境变量找到所需要的值。

相关的指示说明，可以参阅使用 Secret 数据定义容器变量。

需要注意的是，Pod 中环境变量名称允许的字符范围是有限的。如果某些变量名称不满足这些规则，则即使 Pod 是可以启动的，你的容器也无法访问这些变量。

容器镜像拉取 Secret {#using-imagepullsecrets}

如果你尝试从私有仓库拉取容器镜像，你需要一种方式让每个节点上的 kubelet 能够完成与镜像库的身份认证。你可以配置**镜像拉取 Secret**来实现这点。Secret 是在 Pod 层面来配置的。

使用 imagePullSecrets {#using-imagepullsecrets-1}

imagePullSecrets 字段是一个列表，包含对同一名字空间中 Secret 的引用。你可以使用 imagePullSecrets 将包含 Docker（或其他）镜像仓库密码的 Secret 传递给 kubelet。kubelet 使用此信息来替 Pod 拉取私有镜像。参阅 PodSpec API 进一步了解 imagePullSecrets 字段。

手动设定 imagePullSecret {#manually-specifying-an-imagepullsecret}

你可以通过阅读容器镜像 文档了解如何设置 imagePullSecrets。

设置 imagePullSecrets 为自动挂载 {#arranging-for-imagepullsecrets-to-be-automatically-attached}

你可以手动创建 imagePullSecret，并在一个 ServiceAccount 中引用它。对使用该 ServiceAccount 创建的所有 Pod，或者默认使用该 ServiceAccount 创建的 Pod 而言，其 imagePullSecrets 字段都会设置为该服务账号。请阅读向服务账号添加 ImagePullSecret 来详细了解这一过程。

在静态 Pod 中使用 Secret {#restriction-static-pod}

你不可在 中使用 ConfigMap 或 Secret。

使用场景 {#use-case}

使用场景：作为容器环境变量 {#use-case-as-container-environment-variables}

你可以创建 Secret 并使用它为容器设置环境变量。

使用场景：带 SSH 密钥的 Pod {#use-case-pod-with-ssh-keys}

创建包含一些 SSH 密钥的 Secret：

```
kubectrl create secret generic ssh-key-secret
--from-file=ssh-privatekey=/path/to/.ssh/id_rsa
--from-file=ssh-publickey=/path/to/.ssh/id_rsa.pub
```

输出类似于：

secret "ssh-key-secret" created

你也可以创建一个 kustomization.yaml 文件，在其 secretGenerator 字段中包含 SSH 密钥。

在提供你自己的 SSH 密钥之前要仔细思考：集群的其他用户可能有权访问该 Secret。

你也可以创建一个 SSH 私钥，代表一个你希望与你共享 Kubernetes 集群的其他用户分享的服务标识。当凭据信息被泄露时，你可以收回该访问权限。

现在你可以创建一个 Pod，在其中访问包含 SSH 密钥的 Secret，并通过卷的方式来使用它：

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-test-pod
  labels:
    name: secret-test
spec:
  volumes:
  - name: secret-volume
    secret:
      secretName: ssh-key-secret
  containers:
  - name: ssh-test-container
    image: mySshImage
    volumeMounts:
    - name: secret-volume
      readOnly: true
      mountPath: "/etc/secret-volume"
```

容器命令执行时，密钥的数据可以在下面的位置访问到：

```
/etc/secret-volume/ssh-publickey
/etc/secret-volume/ssh-privatekey
```

容器就可以随便使用 Secret 数据来建立 SSH 连接。

使用场景：带有生产、测试环境凭据的 Pod {#use-case-pods-with-prod-test-credentials}

这一示例所展示的一个 Pod 会使用包含生产环境凭据的 Secret，另一个 Pod 使用包含测试环境凭据的 Secret。

你可以创建一个带有 secretGenerator 字段的 kustomization.yaml 文件或者运行 `kubectl create secret` 来创建 Secret。

```
kubectl create secret generic prod-db-secret --from-literal=username=produser --from-literal=password=Y4nys7f11
```

输出类似于：

```
secret "prod-db-secret" created
```

你也可以创建一个包含测试环境凭据的 Secret：

```
kubectl create secret generic test-db-secret --from-literal=username=testuser --from-literal=password=iluvtests
```

输出类似于：

```
secret "test-db-secret" created
```

特殊字符（例如 `$`、`\`、`*`、`=` 和 `!`）会被你的 Shell 解释，因此需要转义。

在大多数 Shell 中，对密码进行转义的最简单方式是用单引号（`'`）将其括起来。例如，如果你的实际密码是 `S!B\*d$zDsb`，则应通过以下方式执行命令：

```
kubectl create secret generic dev-db-secret --from-literal=username=devuser --from-literal=password='S!B\*d$zDsb'
```

你无需对文件中的密码（`--from-file`）中的特殊字符进行转义。

现在生成 Pod：

```
cat kustomization.yaml
resources:
- pod.yaml
EOF
```

通过下面的命令在 API 服务器上应用所有这些对象：

```
kubectl apply -k .
```

两个文件都会在其文件系统中出现下面的文件，文件中内容是各个容器的环境值：

```
/etc/secret-volume/username  
/etc/secret-volume/password
```

注意这两个 Pod 的规约中只有一个字段不同。这便于基于相同的 Pod 模板生成具有不同能力的 Pod。

你可以通过使用两个服务账号来进一步简化这一基本的 Pod 规约：

1. prod-user 服务账号使用 prod-db-secret
2. test-user 服务账号使用 test-db-secret

Pod 规约简化为：

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: prod-db-client-pod  
  labels:  
    name: prod-db-client  
spec:  
  serviceAccount: prod-db-client  
  containers:  
  - name: db-client-container  
    image: myClientImage
```

不可更改的 Secret {#secret-immutable}

Kubernetes 允许你将特定的 Secret（和 ConfigMap）标记为 **不可更改（Immutable）**。禁止更改现有 Secret 的数据有下列好处：

- 防止意外（或非预期的）更新导致应用程序中断
- （对于大量使用 Secret 的集群而言，至少数万个不同的 Secret 供 Pod 挂载），通过将 Secret 标记为不可变，可以极大降低 kube-apiserver 的负载，提升集群性能。kubelet 不需要监视那些被标记为不可更改的 Secret。

将 Secret 标记为不可更改 {#secret-immutable-create}

你可以通过将 Secret 的 immutable 字段设置为 true 创建不可更改的 Secret。例如：

```
apiVersion: v1  
kind: Secret
```



```
metadata:  
...  
data:  
...  
immutable: true
```

你也可以更改现有的 Secret，令其不可更改。

一旦一个 Secret 或 ConfigMap 被标记为不可更改，撤销此操作或者更改 data 字段的内容都是**不可能的**。只能删除并重新创建这个 Secret。现有的 Pod 将维持对已删除 Secret 的挂载点 -- 建议重新创建这些 Pod。

Secret 的信息安全问题 {#information-security-for-secrets}

尽管 ConfigMap 和 Secret 的工作方式类似，但 Kubernetes 对 Secret 有一些额外的保护。

Secret 通常保存重要性各异的数值，其中很多都可能会导致 Kubernetes 中（例如，服务账号令牌）或对外部系统的特权提升。即使某些个别应用能够推导它期望使用的 Secret 的能力，同一名字空间中的其他应用可能会让这种假定不成立。

授权配置会影响命名空间内 Secret 数据的访问方式。例如，授予 Secret 的 **list** 或 **watch** 权限，将允许主体读取该命名空间中的所有 Secret 数据，而不仅仅是其 Pod 显式引用的 Secret。你应将访问权限限制在工作负载运行所需的最小权限集，并避免授予诸如 cluster-admin 之类的宽泛角色，除非出于管理目的需要。

另请参阅授权文档

只有当某个节点上的 Pod 需要某 Secret 时，对应的 Secret 才会被发送到该节点上。如果将 Secret 挂载到 Pod 中，kubelet 会将数据的副本保存在在 tmpfs 中，这样机密的数据不会被写入到持久性存储中。一旦依赖于该 Secret 的 Pod 被删除，kubelet 会删除来自于该 Secret 的机密数据的本地副本。

同一个 Pod 中可能包含多个容器。默认情况下，你所定义的容器只能访问默认 ServiceAccount 及其相关 Secret。你必须显式地定义环境变量或者将卷映射到容器中，才能为容器提供对其他 Secret 的访问。

针对同一节点上的多个 Pod 可能有多个 Secret。不过，只有某个 Pod 所请求的 Secret 才有可能对 Pod 中的容器可见。因此，一个 Pod 不会获得访问其他 Pod 的 Secret 的权限。

配置 Secret 资源的最小特权访问

为了增强 Secrets 的安全措施，使用单独的命名空间来隔离对挂载 Secret 的访问。

在一个节点上以 `privileged: true` 运行的所有容器可以访问该节点上使用的 Secret。

- 有关管理和提升 Secret 安全性的指南，请参阅 [Kubernetes Secret 良好实践](#)
- 学习如何使用 `kubectl` 管理 Secret
- 学习如何使用配置文件管理 Secret
- 学习如何使用 `kustomize` 管理 Secret
- 阅读 API 参考了解 Secret